

CSCE 649 PHYSICALLY-BASED MODELING · MODULE 04 · LECTURE NOTES

Constraints

Enforcing what must be true, instead of pushing toward it.

MODULE OUTCOMES

- Distinguish hard constraints from penalty forces.
- Formulate constraints with Lagrange multipliers.
- Implement Gauss–Seidel constraint projection.
- Explain position-based dynamics and why it dominates in practice.
- Explain what XPBD fixes about PBD.

1 Two ways to enforce a rule

	Penalty forces	Hard constraints
Method	Add a stiff spring that pushes back when the condition is violated.	Solve for the force that makes the condition hold exactly.
Simplicity	Trivial — it is just another force generator.	Requires solving a system of equations each step.
Exactness	Never exact. There is always a residual violation, because the restoring force is proportional to it.	Exact , to solver tolerance.
Time step	Stiff spring forces a tiny step (Module 02).	No stiffness, so no stiffness-induced limit.
Parameters	A stiffness with no physical meaning, tuned by hand.	None, for a hard constraint.

Module 03 used penalty forces throughout, and ran directly into their limitation: cloth that should barely stretch needs a stiffness that makes the simulation unaffordable. Constraints are the way out.

$$\mathbf{C}(\mathbf{x}) = 0 \quad (\text{equality}) \qquad \mathbf{C}(\mathbf{x}) \geq 0 \quad (\text{inequality})$$

A constraint is a scalar function of the state that must be zero (or non-negative). The inextensible rod is $C = |\mathbf{p}_1 - \mathbf{p}_2| - L$. Non-penetration is $C = (\mathbf{p} - \mathbf{q}) \cdot \mathbf{n} \geq 0$. **Inequality constraints are harder** because they are active only sometimes, and determining which are active is itself part of the problem — which is why contact (Module 06) is the difficult case.

THE CONSTRAINT FORCE ACTS ALONG THE GRADIENT

A constraint force exists to enforce a geometric condition, not to add or remove energy. **So it must do no work**, which means it must be perpendicular to every motion the constraint permits.

The directions the constraint permits are exactly those along which C does not change — the tangent space of the constraint surface. Perpendicular to that is the gradient. **So the constraint force must be parallel to ∇C .**

$\mathbf{F} = \lambda \nabla C$, where λ is a single unknown scalar per constraint: the **Lagrange multiplier**. The direction is determined by geometry; only the magnitude has to be solved for, which is an enormous reduction in the size of the problem.

λ is **physically meaningful**. For a contact constraint it is the normal force, which is precisely the quantity Coulomb friction needs (Module 06). For a joint it is the force the joint is transmitting, which is what tells you when it should break.

2 Solving

$$\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T \boldsymbol{\lambda} = -(\mathbf{J} \mathbf{M}^{-1} \mathbf{F}_{\text{ext}} + \mathbf{J} \mathbf{q})$$

Stacking every constraint's gradient as a row of the Jacobian $\mathbf{J}$ gives a linear system for all the multipliers simultaneously. The matrix is sparse, symmetric, and positive semi-definite, so conjugate gradient applies. **This is the correct solution**, and assembling and solving it every step is expensive enough that almost nobody does it in graphics.

```
for (int iter = 0; iter < N_ITER; iter++)
  for (auto& c : constraints) {
    double cval = c.evaluate(p);
    if (c.inequality && cval >= 0) continue;          // inactive
    auto grad = c.gradient(p);
    double wsum = 0;
    for (int i : c.particles) wsum += inv_mass[i] * grad[i].sqnorm();
    if (wsum < 1e-12) continue;                        // everything pinned
    double s = cval / wsum;
    for (int i : c.particles) p[i] -= s * inv_mass[i] * grad[i];
  }
```

Gauss–Seidel projection satisfies the constraints one at a time, repeatedly, each projection using the positions updated by the previous one. It converges to the true solution as the iteration count grows, and in practice four to twenty iterations are used and the residual error accepted.

Note the inverse mass appearing again: the correction is distributed between the participating particles in proportion to their inverse masses, so a heavy particle moves less and a pinned particle (inverse mass zero) does not move at all — with no special case anywhere.

ITERATION COUNT BECOMES A STIFFNESS DIAL, AND THAT IS A PROBLEM

With four iterations, the constraints are only approximately satisfied and the cloth stretches noticeably. With forty, they are nearly exact and the cloth is almost inextensible.

So the material's stiffness is determined by a solver setting rather than by a material parameter. The same cloth at four and forty iterations is two different fabrics.

It also depends on the time step, which is worse: the same simulation run at 30 Hz and 60 Hz produces materials of different stiffness. Changing the frame rate changes the physics.

This is position-based dynamics's central theoretical flaw, and XPBD (§4) exists to fix it. It is also, perversely, part of why PBD became popular: the dial is intuitive to an artist, responds predictably, and never explodes however far it is turned.

3 Position-based dynamics

```
void step(double dt) {
    for (int i = 0; i < n; i++) {                // 1. predict
        v[i] += dt * inv_mass[i] * f_ext[i];
        pred[i] = p[i] + dt * v[i];
    }
    for (int it = 0; it < iters; it++)            // 2. project
        for (auto& c : constraints) c.project(pred, inv_mass);
    for (int i = 0; i < n; i++) {                // 3. derive velocity
        v[i] = (pred[i] - p[i]) / dt;
        p[i] = pred[i];
    }
}
```

STEP 3 IS THE IDEA, AND IT IS WHY PBD CANNOT EXPLODE

In a conventional simulator, velocity is integrated from forces and position is integrated from velocity. An error in the forces becomes an error in velocity, which becomes a larger error in position, which produces larger forces — the feedback loop that makes explicit methods unstable.

PBD inverts this. Positions are *projected* onto valid configurations, and **velocity is then derived from how far the position actually moved**. Velocity is an output of the step, not something that was integrated.

A projection cannot send a point to infinity — it moves it to a configuration satisfying the constraint, which is by construction a sensible place. And since velocity is computed from a bounded position change divided by the step, it too is bounded.

There is no stiffness parameter, so there is no stability limit. Any time step, any material parameters, any configuration — PBD will produce *something*. For an interactive application that is worth more than physical fidelity: a game that is 10% wrong ships, and a game that explodes when a player wedges a crate into a doorway does not.

Cost	Detail
Stiffness is not a material property.	It depends on iteration count and time step. §2.
Artificial damping.	The projection step silently removes energy, and the amount depends on the solver settings. Bouncy things are hard to make bouncy.
Order dependence.	Gauss–Seidel uses updated positions immediately, so the result depends on the order constraints are visited in. Deterministic, but biased — constraints solved last are satisfied best.
Poor convergence on long chains.	Information propagates one constraint per iteration, so a chain of 100 links needs 100 iterations for a disturbance at one end to reach the other. A hanging rope visibly stretches near the anchor.
It is not a physical method.	You cannot derive PBD from Newtonian mechanics. It is a procedure that produces plausible motion, and its parameters do not correspond to measurable quantities.

And it is in every major game engine, because none of these defects matter as much as the guarantee of never exploding. This is Module 01's ordering — stability over accuracy — in its most extreme form, and it is worth being clear-eyed about: the theoretically indefensible method won on engineering grounds.

4 XPBD

COMPLIANCE MAKES STIFFNESS REAL AGAIN

XPBD (extended position-based dynamics, Müller et al. 2016) reintroduces **compliance** $\alpha = 1/k$ — the inverse of stiffness — as an explicit per-constraint parameter, and accumulates the Lagrange multiplier across solver iterations rather than discarding it.

The result converges to a well-defined solution determined by α , and independent of the iteration count and the time step. More iterations now improve *convergence* toward a fixed answer, rather than changing what the answer is.

$\alpha = 0$ **recovers an exactly hard constraint**; larger values give progressively softer material, specified in physically meaningful units that transfer between resolutions and frame rates.

The change is small — a handful of extra lines per constraint and one accumulator — and it removes PBD's worst defect at essentially no cost. **There is no reason to implement plain PBD today.**

5 Making solvers behave

- **Substep rather than iterate.** Given a fixed budget, taking ten substeps with one solver iteration each beats one step with ten iterations — substantially, and for materials across the whole stiffness range. This was the central finding of Müller et al.'s 2019 'small steps' paper, and it is the most valuable practical result in this module.
- **Vary the constraint ordering** between iterations, by reversing or randomising. Gauss–Seidel's sequential nature biases the result toward constraints solved last, and alternating the order spreads that bias.
- **Use graph colouring to parallelise.** Constraints that share no particles are independent and can be projected simultaneously. Colouring the constraint graph identifies these sets, which is how PBD runs on a GPU.
- **Warm start.** Begin each step from the previous step's multipliers rather than from zero. For stacking and resting contact this is a large improvement, because the correct answer changes little between frames.
- **Handle over-constrained systems gracefully.** A particle pinned in two incompatible places has no valid configuration. The solver must settle into a compromise rather than oscillating or diverging — and this situation arises constantly in practice, usually from contact.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Baraff — Constrained Dynamics (free notes)	Section F of the course pack. The Lagrange multiplier formulation of §1 and the global solve of §2, derived properly.
Müller et al. — Position Based Dynamics (2007, free)	The original PBD paper. Short, and the algorithm is exactly §3.
Macklin, Müller & Chentanez — XPBD (2016, free)	The compliance formulation of §4. Read it immediately after the PBD paper.
Müller et al. — Small Steps in Physics Simulation (2019, free)	The substepping result in §5. A short paper with a large practical payoff.
Erin Catto — Soft Constraints and Sequential Impulses (GDC, free)	The same ideas from the game-engine side, and the clearest explanation of warm starting.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a distance constraint by penalty force and by projection. Compare the residual violation and the maximum stable time step for each.
- 2 Derive the gradient of the distance constraint by hand and verify it numerically by finite differences.
- 3 Implement Gauss–Seidel projection for a chain of distance constraints. Plot the residual against iteration count.
- 4 Build a hanging chain of 100 links and show the convergence problem: plot stretch along the chain for 1, 10, and 100 iterations.
- 5 Demonstrate the stiffness problem: simulate the same cloth with 4, 10, and 40 iterations and measure the maximum stretch in each case.
- 6 Repeat with the time step halved and show that the effective stiffness changes.
- 7 Implement XPBD and repeat both experiments. Confirm the stretch now depends on compliance rather than on iterations or step size.
- 8 Compare ten substeps with one iteration against one step with ten iterations, at equal cost. Report which is stiffer and which is more stable.
- 9 Implement graph colouring for a cloth constraint set and report how many colours are needed and how large each independent set is.
- 10 Construct an over-constrained system and show how your solver behaves. Then make it degrade gracefully.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compare penalty forces and hard constraints on four axes.
- 2 Why must a constraint force act along the gradient of C ?
- 3 What is a Lagrange multiplier, and what does it mean physically for a contact?
- 4 Describe Gauss–Seidel projection and the role of inverse mass in it.
- 5 Why does iteration count act as a stiffness dial in PBD, and why is that a problem?
- 6 Write the three steps of PBD and explain why step 3 makes it unconditionally stable.
- 7 Give five costs of PBD, and say why it dominates anyway.
- 8 What does XPBD add, and what does it fix?
- 9 Why does substepping beat iterating, and what else improves a constraint solver?